ITCS213 Computer Organization and Architecture

Class: 1-2026
Sec 2 Fri 9:00-12:00 IT312 (Aj. Joke)
Sec 3 Fri 13:00-16:00 IT312 (Aj. Joke)
Sec 1 Fri 13:00-16:00 IT311 (Aj. Pete)

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Lecturer Dr. Kanoksak Wattanachote (Ph.D) (金功勳 博士)
kanoksak.wat@mahidol.ac.th

Grading

ITCS213 Computer Organization and Architecture

- Overall Score Parts
  - Class activities + Assignments + Quizzes 20%
  - Midterm Exam 40%
  - Final Exam 40%

GRADING SYSTEM

Grade Criteria	A	>=	80
	B+	>=	75
	B	>=	70
	C+	>=	65
	C	>=	60
	D+	>=	55
	D	>=	50
	F	<	50

Content

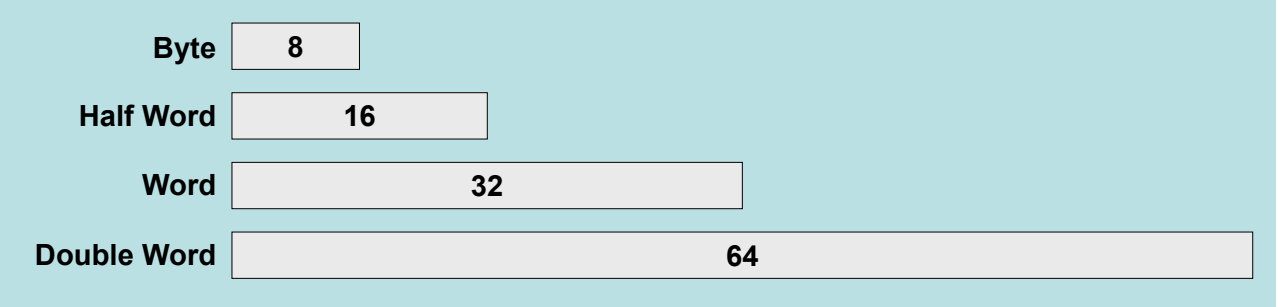
Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- R-Type Arithmetic, Logical, and Shift Instructions
- I-Type Instruction Format and Immediate Constants
- I-Type Arithmetic and Logical Instructions
- Pseudo Instructions
- Integer Multiplication in MIPS

Positional Number System and Integer

Integer Storage Sizes

Unit 7: Logic Bitwise, Shift, and I-Type Instructions



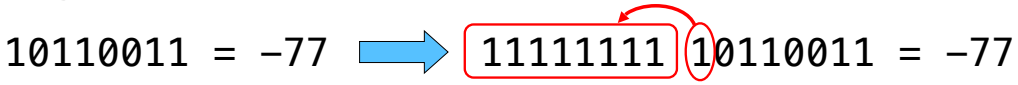
Storage Type	Unsigned Range	Powers of 2
Byte	0 to 255	0 to $(2^8 - 1)$
Half Word	0 to 65,535	0 to $(2^{16} - 1)$
Word	0 to 4,294,967,295	0 to $(2^{32} - 1)$
Double Word	0 to 18,446,744,073,709,551,615	0 to $(2^{64} - 1)$

- What is the largest 20-bit unsigned integer?
- Answer: $2^{20} - 1 = 1,048,575$

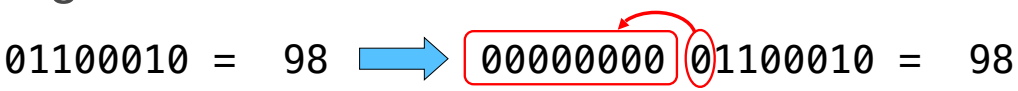
Sign Extension

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Step 1: Move the number into the lower-significant bits
- Step 2: Fill all the remaining higher bits with the sign bit
  - This will ensure that both magnitude and sign are correct
  - Examples
    - Sign-Extend `10110011` to 16 bits:



    - Sign-Extend `01100010` to 16 bits:



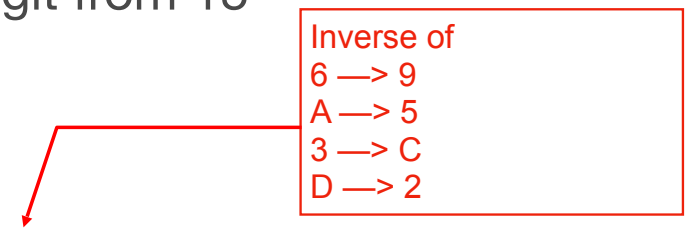
- Infinite 0s can be added to the left of a positive number
- Infinite 1s can be added to the left of a negative number

Two's Complement of a Hexadecimal

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- To form the two's complement of a hexadecimal:
  - Subtract each hexadecimal digit from 15
  - Add 1

- Examples:
  - 2's complement of 6A3D = 95C2 + 1 = 95C3
  - 2's complement of 92F15AC0 = 6D0EA53F + 1 = 6D0EA540
  - 2's complement of FFFFFFFF = 00000000 + 1 = 00000001

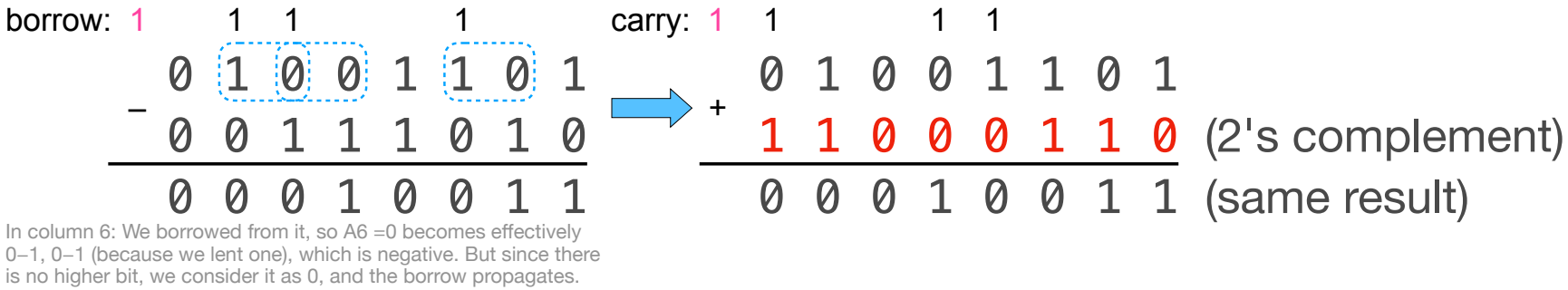


- No need to convert hexadecimal to binary

Binary Subtraction

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- When subtracting A – B, convert B to its 2's complement.
- Add A to (–B)



In column 6: We borrowed from it, so A6 =0 becomes effectively 0–1, 0–1 (because we lent one), which is negative. But since there is no higher bit, we consider it as 0, and the borrow propagates.

- Final carry is ignored because
  - Negative number is sign-extended with 1's.
  - You can imagine infinite 1's to the left of a negative number.
  - Adding the carry to the extended 1's produces extended zeros.

Range of Signed Integers

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- For n-bit signed integers: Range is -2^{n-1} to $(2^{n-1} - 1)$
- Positive range: 0 to $2^{n-1} - 1$
- Negative range: -2^{n-1} to -1

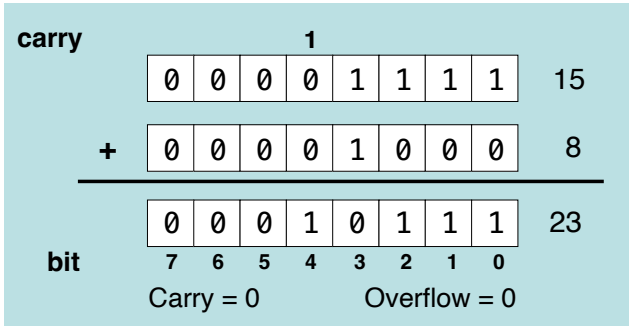
Storage Type	Signed Range	Powers of 2
Byte	–128 to 127	-2^7 to $(2^7 - 1)$
Half Word	–32,768 to 32,767	-2^{15} to $(2^{15} - 1)$
Word	–2,147,483,648 to 2,147,483,647	-2^{31} to $(2^{31} - 1)$
Double Word	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	-2^{63} to $(2^{63} - 1)$

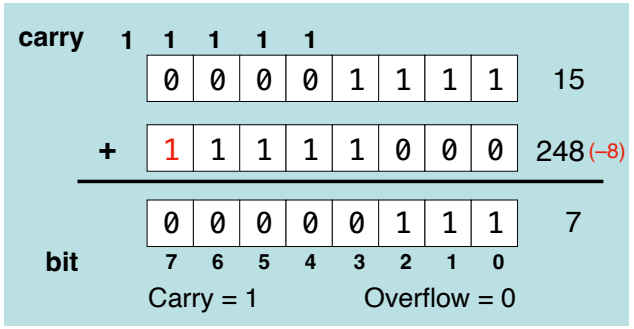
- Practice: What is the range of signed values that may be stored in 26 bits?

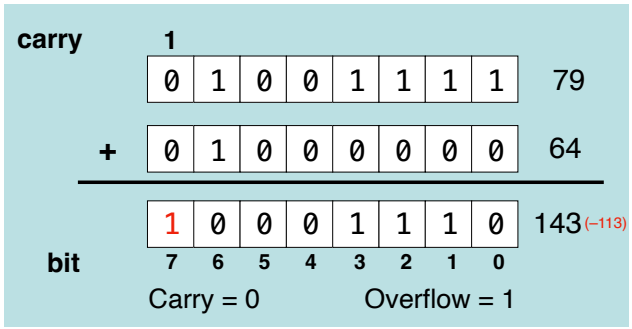
Carry and Overflow Examples

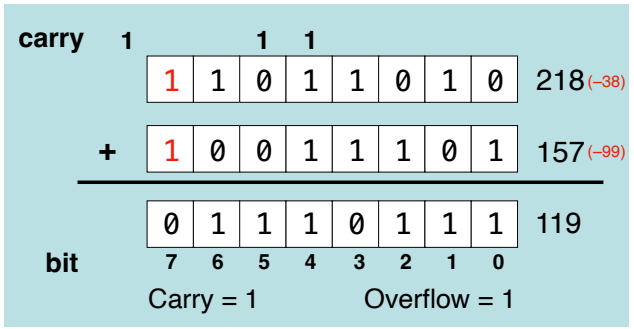
Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- We can have carry without overflow and vice versa.
- Four cases are possible (Examples are 8-bit numbers).









Printable ASCII Codes

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Examples:
  - ASCII code for space character = 20 (hex) = 32 (decimal).
  - ASCII code for 'L' = 4C (hex) = 76 (decimal).
  - ASCII code for 'a' = 61 (hex) = 97 (decimal).

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2	space	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Control Characters

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- The first 32 characters of the ASCII table are used for control.
- Control character codes = 00 to 1F (hexadecimal)
    - Not shown in the previous slide.
- Examples of Control Characters:
    - Character 0 is the NULL character ⇒ used to terminate a string.
    - Character 9 is the Horizontal Tab (HT) character.
    - Character 0A (hex) = 10 (decimal) is the Line Feed (LF).
    - Character 0D (hex) = 13 (decimal) is the Carriage Return (CR).
    - The LF and CR characters are used together.
        - They advance the cursor to the beginning of the next line.
- One control character appears at the end of the ASCII table.
    - Character 7F (hex) is the Delete (DEL) character.

Carry, Borrow, Overflow, and Underflow (+ Zero)

Definitions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System

- Carry: Occurs during unsigned addition when the result exceeds the maximum value, e.g., >15 in a 4-bit.
    - It's the bit carried out from the Most Significant Bit (MSB).
- Borrow: Occurs during unsigned subtraction when the result would be negative, e.g., <0 in a 4-bit.
    - It indicates a borrow was needed from a higher bit.

Definitions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System

- Overflow: Occurs during signed addition or subtraction when the result is outside the representable range, e.g., not between -8 and 7 in a 4-bit.
    - It occurs when two positives add to a negative, two negatives add to a positive, and so on.
- Underflow: This term is often used interchangeably with Borrow in the context of unsigned integers (meaning the result went below zero).
    - It can also describe a signed result that is too negative (a type of overflow); however, it's most clearly understood as the unsigned counterpart to the Carry.

Carry, Borrow, Overflow, and Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

For –2 —> 2 reverse

```
0b1110
0b0001 +1
0b0010
```

4-bit System A-B—>A+(-B)

Operation	Bit Calculation	Unsigned View & Flags	Signed View & Flags	Evidence for …
15 + 1	1111 + 0001 = (1)0000	15 + 1 = 16 —> 0 Carry = 1 Zero = 1	–1 + 1 = 0 —> 0 Overflow = 0 Zero = 1	Carry
3 – 5	0011 + 1011 = 1110 0011 – 0101 =–0010	3 – 5 = –2 —> 14 Borrow = 1 (Underflow)	3 – 5 = –2 —> –2 Overflow = 0	Borrow / Underflow (unsigned)
5 + 5	0101 + 0101 = 1010	5 + 5 = 10 —> 10 Carry = 0	5 + 5 = 10 —> –6 Overflow = 1	Overflow
–8 – 1	1000 + 1111 = (1)0111 1000 – 0001 = 0111	8 + 15 = 23 —> 7 Borrow = 0 Carry = 1	–8 – 1 = –9 —> 7 Overflow = 1	Overflow / Underflow (signed)

This situation is handled by **wrapping around** modulo 16.

```
  0011 –          1000 –
  0101            0001
  ————            ————
(1)1011 –         0110 –
  0101            0001
  1110            0111
```

We borrow from this `1`. This causes a "**ripple**" effect:

1. The `1` in column 3 becomes `0`.
2. Column 2 becomes `10` (binary for 2), but since it was `0` and we're borrowing, it effectively becomes `1` (after lending to column 1).
3. Column 1 becomes `10`, but then also becomes `1` (after lending to column 0).
4. Column 0 becomes `10` (binary 2).

Carry, Borrow, Overflow, and Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System A-B—>A+(-B)

7 + 7

Bit Calculation: `0111 + 0111 = 1110`

```
Signed
0111 +
0111
————
1110(-2) overflow
+(0)->-(1)
```

Aspect	Unsigned View & Flags	Signed View & Flags	Explanation
Calculation	7 + 7 = 14	7 + 7 = 14	The calculation `1110` is 14 in unsigned.
Result	14 (Within 0-15 range)	-2 (14 is outside -8 to +7 range)	The signed interpretation is incorrect because the correct answer (14) is too large for a 4-bit signed representation.
Flags	`Carry = 0` (No carry out beyond bit 3)	`Overflow = 1` (Two positives added to give a negative)	The Overflow Flag (`OF=1`) is set because the sign of the result is incorrect.

Carry, Borrow, Overflow, and Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System A-B—>A+(-B)

10 - 3

Unsigned
`(1)0111 (7) carry`

Signed
```
 1010 +
 1101
 ————
10111(-9)
01001 (9)
(1)0111 (7) overflow
-(1)->+(0)
```

Bit Calculation (using addition of -3): `1010 + 1101 = (1)0111` (Discard the extra carry)
Final Result: `0111`

Aspect	Unsigned View & Flags	Signed View & Flags	Explanation
Calculation	10 - 3 = 7	10 is invalid! Let's pretend it's valid: -6 - 3 = -9	`1010` is -6 (-8+2) in signed. The math is broken from the start.
Result	7 (Valid result)	7 (Invalid result, should be -9)	The operation wrapped around modulo 16. The result `0111` (7) is correct for unsigned but nonsense for signed.
Flags	`Carry = 1` (A carry out occurred, indicating no borrow was needed)	`Overflow = 1` (The result makes no sense)	The Carry Flag (`CF=1`) confirms the result is valid for unsigned. The Overflow Flag (`OF=1`) warns the signed result is garbage.

Carry, Borrow, Overflow, and Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System A-B—>A+(-B)

-4 + 2

Unsigned
0100(4)–>1100(–4)=(12)

Bit Calculation: `1100 + 0010 = 1110`

Signed
```
1100 +
0010
————
1110(–2)
```

Aspect	Unsigned View & Flags	Signed View & Flags	Explanation
Calculation	12 + 2 = 14 wrapped around –4—>12	(-4) + 2 = -2	The calculation `1110` is 14 in unsigned and -2 in signed.
Result	14 (Incorrect!) wrapped around 14—>–2	-2 (Correct!)	This is a perfectly valid operation in signed arithmetic.
Flags	`Carry = 0`	`Overflow = 0` (A negative and a positive added correctly)	No flags are set because the operation completed successfully in both systems.

Carry, Borrow, Overflow, and Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System A-B—>A+(-B)

1 - 1

Unsigned
`0001(1)->1111(-1) = (1)0000 carry`

Signed
```
0001 +
1111
-----
(1)0000
```

Bit Calculation (using addition of -1): `0001 + 1111 = (1)0000` (Discard the extra carry)
Final Result: `0000`

Aspect	Unsigned View & Flags	Signed View & Flags	Explanation
Calculation	1 - 1 = 0	1 - 1 = 0	The calculation `0000` is 0 in both systems.
Result	0 (Correct!)	0 (Correct!)	This is a perfectly valid operation.
Flags	`Carry = 1` (Indicates no borrow was needed)	`Overflow = 0`	The Carry Flag is set as expected for unsigned subtraction without underflow. The Zero Flag (`ZF=1`) would also be set.

Carry, Borrow, Overflow, and Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System A-B—>A+(-B)

0 + 15

Unsigned
`1111 (15)`

Bit Calculation: `0000 + 1111 = 1111`

Signed
```
0000 +
1111 (-1)
————
1111 (-1)
0001  (1)
```

Aspect	Unsigned View & Flags	Signed View & Flags	Explanation
Calculation	0 + 15 = 15	0 + (-1) = -1	The calculation `1111` is 15 in unsigned and -1 in signed.
Result	15 (Correct!)	-1 (Correct!)	This is a valid operation in both systems. The results are just interpreted differently.
Flags	`Carry = 0` (No carry out)	`Overflow = 0` (The result is correct for the inputs given)	No flags are set. The operation is successful. But the program gained wrong answer for the signed operation.

Carry, Borrow, Overflow, and Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System A-B—>A+(-B)

7 - 8

Bit Calculation (using addition of -8): `0111 + 1000 = 1111`

Unsigned

```
0111 – 1000 = (10111–1000) –> 1111 (15) –> borrow 1
```

Signed

```
0111 +
1000 (–8)
————–
1111 (–1)
0001 (1)
```

Aspect	Unsigned View & Flags	Signed View & Flags	Explanation
Calculation	7 - 8 = -1 --> 15 (Wraps around)	7 - (-8) = 15	The calculation `1111` is 15 in unsigned and -1 in signed.
Result	15 (Wraps around modulo 16)	15 (…!) = -1	For unsigned, this is an underflow, handled by wrapping. For signed, this is correct.
Flags	`Carry = 0` BUT `Borrow = 1`* /`Underflow(Unsigned)`	`Overflow = 1`	`+15` is outside that range. So, signed overflow occurs: OV = 1.

Flags

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- You cannot rely on ALU flags to detect that kind of program mistake.
- Flags detect arithmetic conditions like carry, borrow, signed overflow, or unsigned underflow.
- They do not detect “you gave me a value that is not representable in the interpretation you wanted,” nor “you are reading the same bits as a different type.
- The ALU only sees 4-bit patterns.
  - It does not know whether 1111 means:
    - unsigned 15
    - signed -1
- The programmer/compiler supplies that meaning.

Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System A-B—>A+(-B)

- Unsigned subtraction → Borrow / Underflow:
  - Happens when A < B for unsigned A - B.
  - The true result is below 0, so it is outside 0..15.

- For example, 2 - 4, is not signed overflow.
  - It is unsigned borrow/underflow.

Underflow

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

4-bit System A-B—>A+(-B)

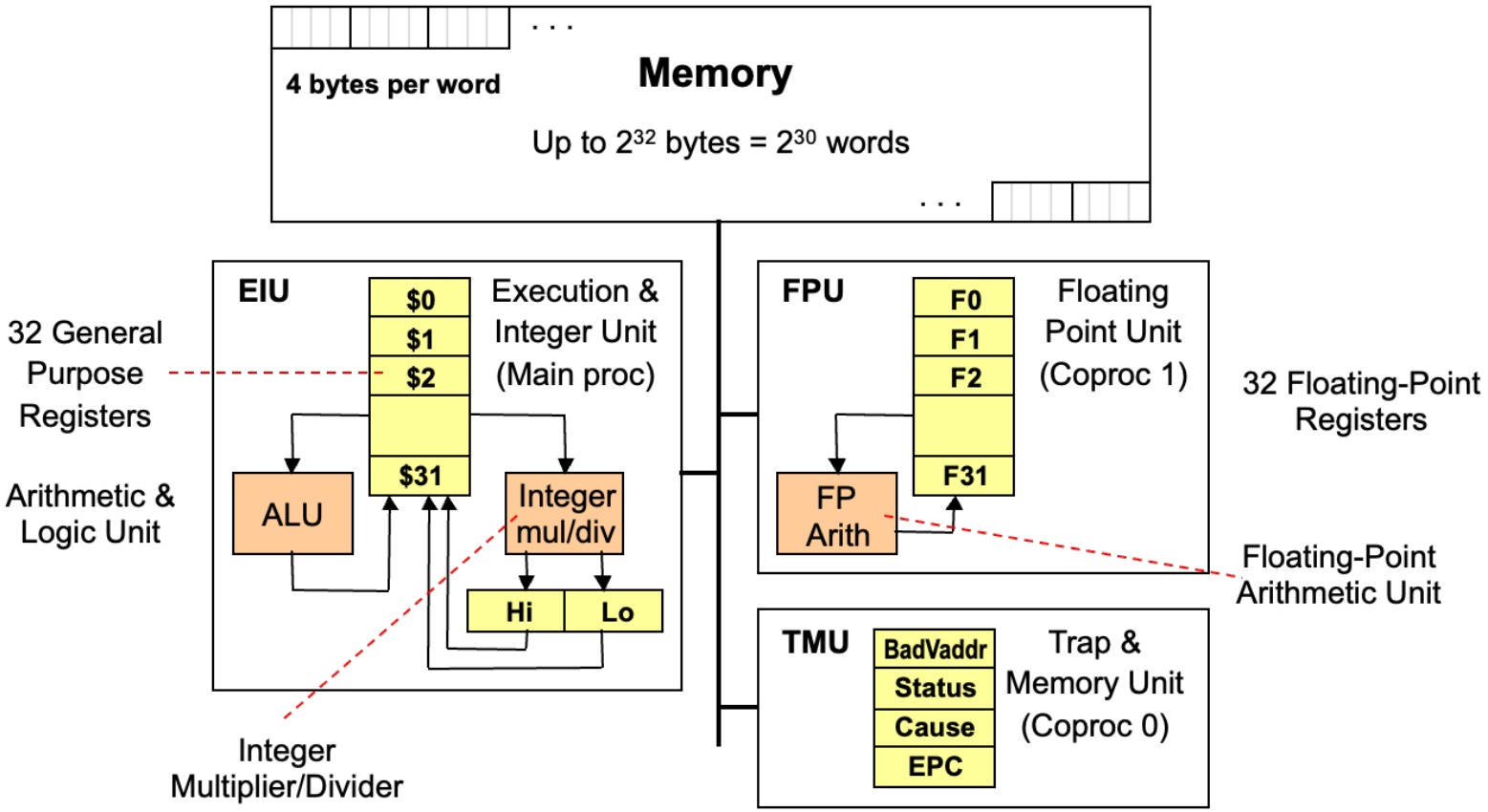
- Signed subtraction → Signed Overflow / Negative Overflow / Underflow:
  - Happens when the signed result is outside -8..7.
  - For -A - B, compute -A + (-B), then check the signed addition overflow rule.

- For example, -8 - 1, is signed overflow.
  - Some people call it signed underflow, but the hardware flag is usually OV = 1.

MIPS General-Purpose Registers

MIPS General-Purpose Registers

Unit 7: Logic Bitwise, Shift, and I-Type Instructions



MIPS General-Purpose Registers

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- 32 General Purpose Registers (GPRs)
  - An assembler uses the dollar notation to name registers
    - $0 is register 0, $1 is register 1, …, and $31 is register 31
  - All registers are 32-bit wide in MIPS32
  - Register $0 is always zero
    - Any value written to $0 is discarded
- Software conventions
  - Software defines names for all registers
    - To standardize their use in programs
  - Example: $8 - $15 are called $t0 - $t7
    - Used for temporary values

$0 = $zero	$16 = $s0
$1 = $at	$17 = $s1
$2 = $v0	$18 = $s2
$3 = $v1	$19 = $s3
$4 = $a0	$20 = $s4
$5 = $a1	$21 = $s5
$6 = $a2	$22 = $s6
$7 = $a3	$23 = $s7
$8 = $t0	$24 = $t8
$9 = $t1	$25 = $t9
$10 = $t2	$26 = $k0
$11 = $t3	$27 = $k1
$12 = $t4	$28 = $gp
$13 = $t5	$29 = $sp
$14 = $t6	$30 = $fp
$15 = $t7	$31 = $ra

MIPS Register Conventions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- An assembler can refer to registers by name or by number.
- It is easier for you to remember registers by name.
- An assembler converts a register name to its corresponding number.

Name	Register	Usage
$zero	$0	Always 0 (forced by hardware)
$at	$1	Reserved for assembler use
$v0 – $v1	$2 – $3	Result values of a function
$a0 – $a3	$4 – $7	Arguments of a function
$t0 – $t7	$8 – $15	Temporary Values
$s0 – $s7	$16 – $23	Saved registers (preserved across call)
$t8 – $t9	$24 – $25	More temporaries
$k0 – $k1	$26 – $27	Reserved for OS kernel
$gp	$28	Global pointer (points to global data)
$sp	$29	Stack pointer (points to top of stack)
$fp	$30	Frame pointer (points to stack frame)
$ra	$31	Return address (used by jal for function call)

Instruction Formats

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- All instructions are 32-bit wide. Three instruction formats:
  - Register (R-Type)
    - Register-to-register instructions.
    - Op: operation code specifies the format of the instruction.



  - Immediate (I-Type)
    - A 16-bit immediate constant is part of the instruction.



  - Jump (J-Type)
    - Used by jump instructions.



R-Type Operation Example

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Consider the translation of: `f = (g+h)-(i+j).`
- A programmer/compiler allocates registers to variables
- Given that: `$t0=f`, `$t1=g`, `$t2=h`, `$t3=i`, and `$t4=j`
- Called temporary registers: `$t0=$8`, `$t1=$9`, …
- Translation of: `f = (g+h)-(i+j)`

```
addu  $t5, $t1, $t2  # $t5 = g + h
addu  $t6, $t3, $t4  # $t6 = i + j
subu  $t0, $t5, $t6  # f = (g+h)-(i+j)
```

- Assembler translates `addu $t5, $t1, $t2` into binary code.

Op	$t1	$t2	$t5	sa	addu
000000	01001	01010	01101	00000	100001

R-Type Operation Example

Unit 7: Logic Bitwise, Shift, and I-Type Instruction



- Translation of: `f = (g+h)-(i+j)`

```
addu  $t5, $t1, $t2  # $t5 = g + h
```

Op	$t1	$t2	$t5	sa	addu
000000	01001	_____	_____	_____	_____

```
addu  $t6, $t3, $t4  # $t6 = i + j
```

Op	$t3	$t4	$t6	sa	addu
000000	_____	_____	_____	_____	_____

```
subu  $t0, $t5, $t6  # f = (g+h)-(i+j)
```

Op	$t5	$t6	$t0	sa	subu
000000	_____	_____	_____	_____	_____

MIPS Special Opcode Encoding of Function

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

function		bits 2..0							
		0	1	2	3	4	5	6	7
bits 5. 3		000	001	010	011	100	101	110	111
0	000	SLL[1]	*MOVCI* [6R]δ	*SRL* δ	SRA	SLLV	LSA[6N]	*SRLV* δ	SRAV
1	001	JR[2,3,6R]	JALR[2]	MOVZ[6R]	MOVN[6R]	SYSCALL	BREAK	SDBBP[6Nm]	SYNC
2	010	MFHI[6R] CLZ[6Nm]	MTHI[6R] CLO[6Nm]	MFLO[6R]	MTLO[6R]	β	β	β	β
3	011[4]	[4]MULT[6R] SOP30[6N]	[4]MULTU[6R] SOP31[6N]	[4]DIV[6R] SOP32[6N]	[4]DIVU[6R] SOP33[6N]	β	β	β	β
4	100	ADD	ADDU	SUB	SUBU	AND	OR	XOR	NOR
5	101	*	*	SLT	SLTU	β	β	β	β
6	110	TGE	TGEU	TLT	TLTU	TEQ	SELEQZ[6N]	TNE	SELNEZ[6N]
7	111	β	*	β	β	β	*	β	β

1. Specific encodings of the *rt*, *rd*, and *sa* fields are used to distinguish among the SLL, NOP, SSNOP, EHB and PAUSE functions. Release 6 makes SSNOP equivalent to NOP.
2. Specific encodings of the *hint* field are used to distinguish JR from JR.HB and JALR from JALR.HB

Assembly Language Statements

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Typically, one statement should appear on a line.
- Three types of statements in assembly language:
  1. Executable Instructions
     - Generate machine code for the processor to execute at runtime.
     - Instructions tell the processor what to do.
  2. Pseudo-Instructions and Macros
     - Translated by the assembler into real instructions.
     - Simplify the programmer's task.
  3. Assembler Directives
     - Provide information to the assembler while translating a program.
     - Used to define segments, allocate memory variables, etc.
     - Non-executable: directives are not part of the instruction set.

Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Assembly language instructions have the format:

  `[label:]  mnemonic  [operands]  [#comment]`

  - Label: (optional)
    - Marks the address of a memory location, must have a colon.
    - Typically appear in data and text segments.
  - Mnemonic
    - Identifies the operation, e.g., add, sub, etc.
  - Operands
    - Specify the data required by the operation
    - Operands can be registers, memory variables, or constants
    - Most instructions have three operands
- L1: addiu $t0, $t0, 1 #increment $t0

Logic Bitwise Operations

MIPS Special Opcode Encoding of Function

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

function		bits 2..0							
		0	1	2	3	4	5	6	7
bits 5. 3		000	001	010	011	100	101	110	111
0	000	SLL[1]	*MOVCI* [6R]δ	*SRL* δ	SRA	SLLV	LSA[6N]	*SRLV* δ	SRAV
1	001	JR[2,3,6R]	JALR[2]	MOVZ[6R]	MOVN[6R]	SYSCALL	BREAK	SDBBP[6Nm]	SYNC
2	010	MFHI[6R] CLZ[6Nm]	MTHI[6R] CLO[6Nm]	MFLO[6R]	MTLO[6R]	β	β	β	β
3	011[4]	[4]MULT[6R] SOP30[6N]	[4]MULTU[6R] SOP31[6N]	[4]DIV[6R] SOP32[6N]	[4]DIVU[6R] SOP33[6N]	β	β	β	β
4	100	ADD	ADDU	SUB	SUBU	AND	OR	XOR	NOR
5	101	*	*	SLT	SLTU	β	β	β	β
6	110	TGE	TGEU	TLT	TLTU	TEQ	SELEQZ[6N]	TNE	SELNEZ[6N]
7	111	β	*	β	β	β	*	β	β

1. Specific encodings of the *rt*, *rd*, and *sa* fields are used to distinguish among the SLL, NOP, SSNOP, EHB and PAUSE functions. Release 6 makes SSNOP equivalent to NOP.
2. Specific encodings of the *hint* field are used to distinguish JR from JR.HB and JALR from JALR.HB

R-Type Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func	
`add $t1, $t2, $t3`	`$t1 = $t2 + $t3`	0	$t2	$t3	$t1	0	0x20	0010 0000
`addu $t1, $t2, $t3`	`$t1 = $t2 + $t3`	0	$t2	$t3	$t1	0	0x21	0010 0001
`sub $t1, $t2, $t3`	`$t1 = $t2 - $t3`	0	$t2	$t3	$t1	0	0x22	0010 0010
`subu $t1, $t2, $t3`	`$t1 = $t2 - $t3`	0	$t2	$t3	$t1	0	0x23	0010 0011

Logic Bitwise Operations

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Logic bitwise operations: `and`, `or`, `xor`, `nor`

x	y	x and y
0	0	0
0	1	0
1	0	0
1	1	1

x	y	x or y
0	0	0
0	1	1
1	0	1
1	1	1

x	y	x xor y
0	0	0
0	1	1
1	0	1
1	1	0

x	y	x nor y
0	0	1
0	1	0
1	0	0
1	1	0

Logic Bitwise Operations

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Logic bitwise operations: `and`, `or`, `xor`, `nor`

x	y	x and y
0	0	0
0	1	0
1	0	0
1	1	1

x	y	x or y
0	0	0
0	1	1
1	0	1
1	1	1

x	y	x xor y
0	0	0
0	1	1
1	0	1
1	1	0

x	y	x nor y
0	0	1
0	1	0
1	0	0
1	1	0

- AND instruction is used to clear bits: `x and 0` → `0`
- OR instruction is used to set bits: `x or 1` → `1`
- XOR instruction is used to toggle bits: `x xor 1` → `not x`
- NOT instruction is not needed, why?
  - `not $t1, $t2` is equivalent to: `nor $t1, $t2, $t2`

The Logic: Understanding NOR

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Logic bitwise operations: `and`, `or`, `xor`, `nor`

x	y	x and y
0	0	0
0	1	0
1	0	0
1	1	1

x	y	x or y
0	0	0
0	1	1
1	0	1
1	1	1

x	y	x xor y
0	0	0
0	1	1
1	0	1
1	1	0

x	y	x nor y
0	0	1
0	1	0
1	0	0
1	1	0

- Now, what happens if we make both inputs the same?
- Let's set A = X and B = X.
- The truth table simplifies to:
  - Because `0 nor 0` is 1 → `not 0` is 1
  - Because 1 `nor` 1 is 0 → `not` 1 is 0
  - Therefore, `x nor x` is logically identical to `not x`.

x	x	x nor x
0	0	1
1	1	0

Logic Bitwise Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func	
`and $t1, $t2, $t3`	`$t1 = $t2 & $t3`	0	$t2	$t3	$t1	0	0x24	0010 0100
`or $t1, $t2, $t3`	`$t1 = $t2 | $t3`	0	$t2	$t3	$t1	0	0x25	0010 0101
`xor $t1, $t2, $t3`	`$t1 = $t2 ^ $t3`	0	$t2	$t3	$t1	0	0x26	0010 0110
`nor $t1, $t2, $t3`	`$t1 = ~($t2 | $t3)`	0	$t2	$t3	$t1	0	0x27	0010 0111

- Examples:
- Given: `$t1` = `0xabcd1234` and `$t2` = `0xffff0000`

```
and $t0, $t1, $t2        # $t0 = 0x________
```

- This performs a bitwise `AND` between `$t1` and `$t2`, storing the result in `$t0`.

```
# $t1 = 0xabcd1234 → Binary: 1010 1011 1100 1101 0001 0010 0011 0100
# $t2 = 0xffff0000 → Binary: 1111 1111 1111 1111 0000 0000 0000 0000
# $t0 = 0x________ → Binary: 1010 1011 1100 1101 0000 0000 0000 0000
```

Logic Bitwise Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func	
`and $t1, $t2, $t3`	`$t1 = $t2 & $t3`	0	$t2	$t3	$t1	0	0x24	0010 0100
`or $t1, $t2, $t3`	`$t1 = $t2 | $t3`	0	$t2	$t3	$t1	0	0x25	0010 0101
`xor $t1, $t2, $t3`	`$t1 = $t2 ^ $t3`	0	$t2	$t3	$t1	0	0x26	0010 0110
`nor $t1, $t2, $t3`	`$t1 = ~($t2 | $t3)`	0	$t2	$t3	$t1	0	0x27	0010 0111

- Examples:
- Given: $t1 = 0xabcd1234 and $t2 = 0xffff0000

```
and $t0, $t1, $t2        # $t0 = 0x_________
or  $t0, $t1, $t2        # $t0 = 0x_________
xor $t0, $t1, $t2        # $t0 = 0x_________
nor $t0, $t1, $t2        # $t0 = 0x_________
```



Logic Bitwise Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func	
`and $t1, $t2, $t3`	`$t1 = $t2 & $t3`	0	$t2	$t3	$t1	0	0x24	0010 0100
`or $t1, $t2, $t3`	`$t1 = $t2 | $t3`	0	$t2	$t3	$t1	0	0x25	0010 0101
`xor $t1, $t2, $t3`	`$t1 = $t2 ^ $t3`	0	$t2	$t3	$t1	0	0x26	0010 0110
`nor $t1, $t2, $t3`	`$t1 = ~($t2 | $t3)`	0	$t2	$t3	$t1	0	0x27	0010 0111

- Examples:
- Given: $t1 = 0xabcdf214 and $t2 = 0xffff0201

```
and $t0, $t1, $t2        # $t0 = 0x_________
or  $t0, $t1, $t2        # $t0 = 0x_________
xor $t0, $t1, $t2        # $t0 = 0x_________
nor $t0, $t1, $t2        # $t0 = 0x_________
```



Logic Bitwise Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Given: `$t1` = `0xabcd1234` and `$t2` = `0xffff0000`

```
and $t0, $t1, $t2        # $t0 = 0x________
```

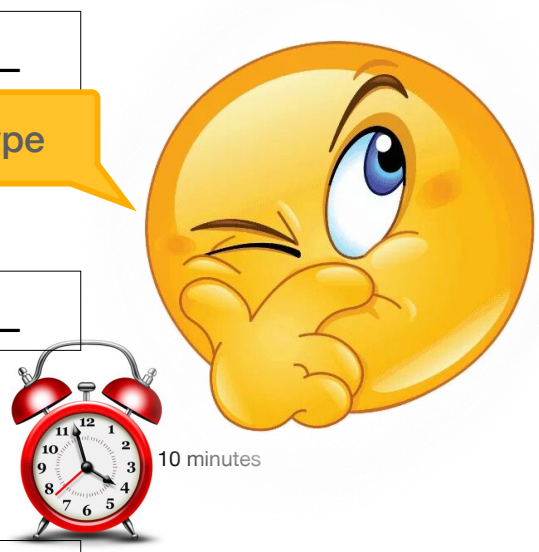
Op	$t1	$t2	$t0	sa	and
000000	_____	_____	_____	_____	_____

```
or  $t0, $t1, $t2        # $t0 = 0x________
```

Op	$t1	$t2	$t0	sa	or
000000	_____	_____	_____	_____	_____

```
xor $t0, $t1, $t2        # $t0 = 0x________
```

Op	$t1	$t2	$t0	sa	xor
000000	_____	_____	_____	_____	_____



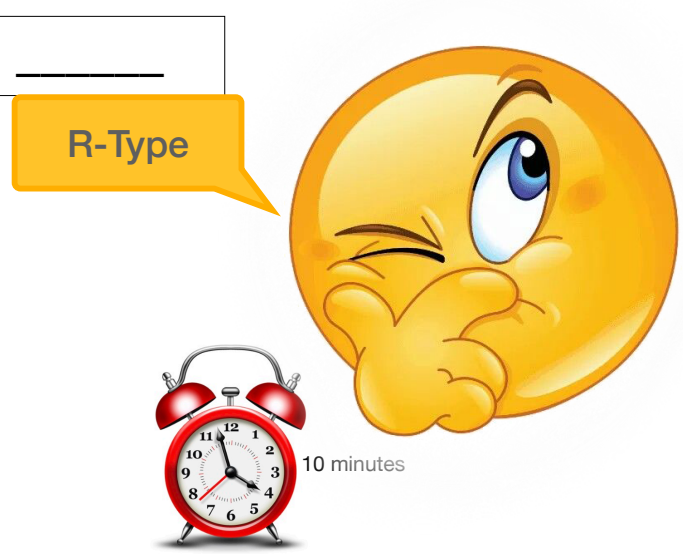
Logic Bitwise Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Given: `$t1` = `0xabcd1234` and `$t2` = `0xffff0000`

```
nor $t0, $t1, $t2        # $t0 = 0x_________
```

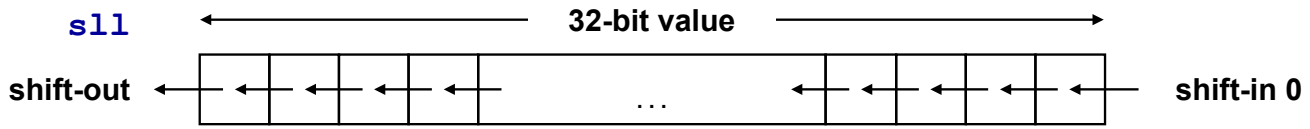
Op	$t1	$t2	$t0	sa	nor
000000	_____	_____	_____	_____	_____



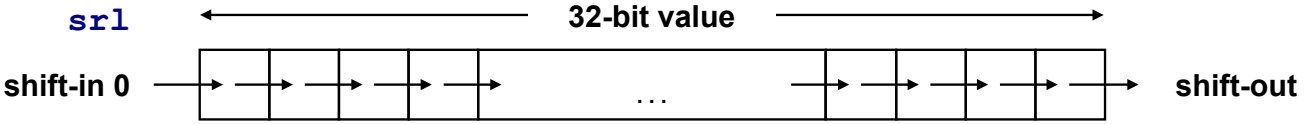
Shift Operations

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

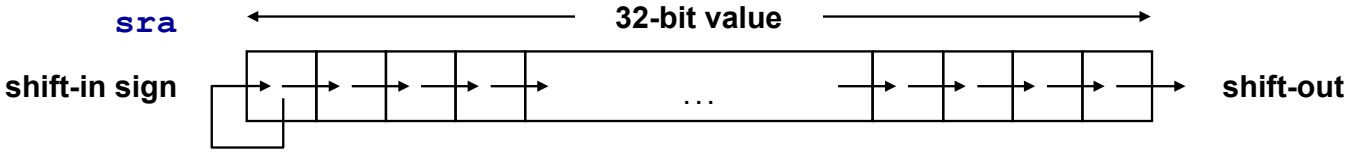
- Shifting is the process of moving the 32 bits of a number left or right.
- These instructions use the 5-bit shift amount field.
- **`sll`** means shift left logical (insert zero from the right).



- **`srl`** means shift right logical (insert zero from the left).



- **`sra`** means shift right arithmetic (insert sign-bit).



MIPS Special Opcode Encoding of Function

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

function		bits 2..0							
		0	1	2	3	4	5	6	7
bits 5. 3		000	001	010	011	100	101	110	111
0	000	SLL[1]	*MOVCI* [6R]δ	*SRL* δ	SRA	SLLV	LSA[6N]	*SRLV* δ	SRAV
1	001	JR[2,3,6R]	JALR[2]	MOVZ[6R]	MOVN[6R]	SYSCALL	BREAK	SDBBP[6Nm]	SYNC
2	010	MFHI[6R] CLZ[6Nm]	MTHI[6R] CLO[6Nm]	MFLO[6R]	MTLO[6R]	β	β	β	β
3	011[4]	[4]MULT[6R] SOP30[6N]	[4]MULTU[6R] SOP31[6N]	[4]DIV[6R] SOP32[6N]	[4]DIVU[6R] SOP33[6N]	β	β	β	β
4	100	ADD	ADDU	SUB	SUBU	AND	OR	XOR	NOR
5	101	*	*	SLT	SLTU	β	β	β	β
6	110	TGE	TGEU	TLT	TLTU	TEQ	SELEQZ[6N]	TNE	SELNEZ[6N]
7	111	β	*	β	β	β	*	β	β

1. Specific encodings of the *rt*, *rd*, and *sa* fields are used to distinguish among the SLL, NOP, SSNOP, EHB and PAUSE functions. Release 6 makes SSNOP equivalent to NOP.
2. Specific encodings of the *hint* field are used to distinguish JR from JR.HB and JALR from JALR.HB

Shift Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func	
`sll $t1,$t2,10`	`$t1 = $t2 << 10`	0	0	$t2	$t1	10	0x0	0000 0000
`srl $t1,$t2,10`	`$t1 = $t2 >> 10`	0	0	$t2	$t1	10	0x2	0000 0010
`sra $t1,$t2,10`	`$t1 = $t2 >> 10`	0	0	$t2	$t1	10	0x3	0000 0011
`sllv $t1,$t2,$t3`	`$t1 = $t2 << $t3`	0	$t3	$t2	$t1	0	0x4	0000 0100
`srlv $t1,$t2,$t3`	`$t1 = $t2 >> $t3`	0	$t3	$t2	$t1	0	0x6	0000 0110
`srav $t1,$t2,$t3`	`$t1 = $t2 >> $t3`	0	$t3	$t2	$t1	0	0x7	0000 0111

- **`sll`**, **`srl`**, **`sra`**: shift by a constant amount.
  - The shift amount (`sa`) field specifies a number between 0 and 31.
- **`sllv`**, **`srlv`**, **`srav`**: shift by a variable amount.
  - A source register specifies the variable shift amount, ranging from 0 to 31.
  - Only the lower 5 bits of the source register are used as the shift amount.

Shift Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func	
`sll $t1,$t2,10`	`$t1 = $t2 << 10`	0	0	$t2	$t1	10	0x0	0000 0000
`srl $t1,$t2,10`	`$t1 = $t2 >> 10`	0	0	$t2	$t1	10	0x2	0000 0010
`sra $t1,$t2,10`	`$t1 = $t2 >> 10`	0	0	$t2	$t1	10	0x3	0000 0011
`sllv $t1,$t2,$t3`	`$t1 = $t2 << $t3`	0	$t3	$t2	$t1	0	0x4	0000 0100
`srlv $t1,$t2,$t3`	`$t1 = $t2 >> $t3`	0	$t3	$t2	$t1	0	0x6	0000 0110
`srav $t1,$t2,$t3`	`$t1 = $t2 >> $t3`	0	$t3	$t2	$t1	0	0x7	0000 0111

- Given that: $t2 = 0xabcd1234 and $t3 = 16

```
sll  $t1, $t2, 8        # $t1 = 0x_________
srl  $t1, $t2, 4        # $t1 = 0x_________
sra  $t1, $t2, 4        # $t1 = 0x_________
srlv $t1, $t2, $t3      # $t1 = 0x_________
```



10 minutes



Shift Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Given that: `$t2` = `0xabcd1234` and `$t3` = 16

```
sll  $t1, $t2, 8          # $t1 = 0x________
```

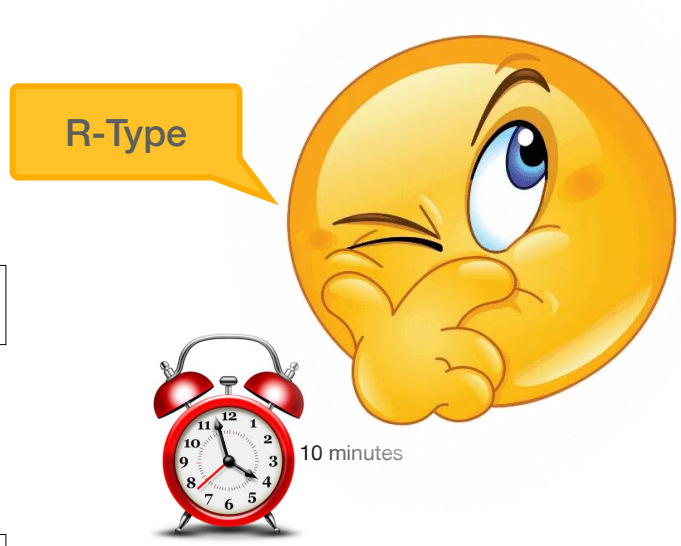
Op	rs	$t2	$t1	sa	sll
000000	_____	_____	_____	_____	_____

```
srl  $t1, $t2, 4          # $t1 = 0x________
```

Op	rs	$t2	$t1	sa	srl
000000	_____	_____	_____	_____	_____

```
sra  $t1, $t2, 4          # $t1 = 0x________
```

Op	rs	$t2	$t1	sa	sra
000000	_____	_____	_____	_____	_____



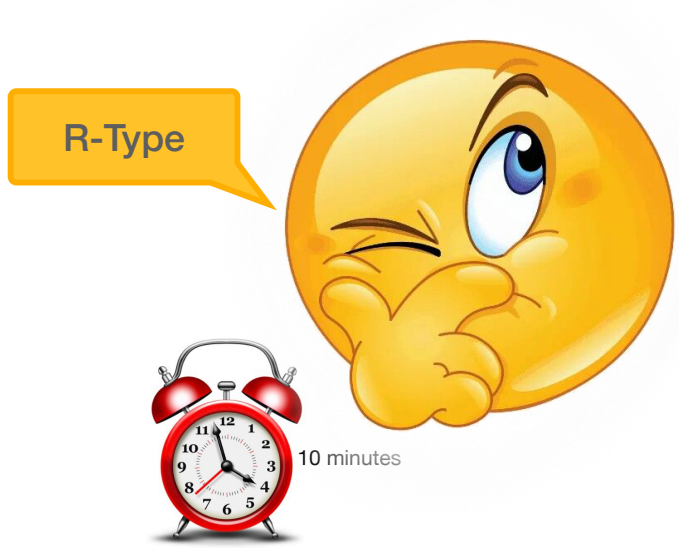
Shift Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Given that: `$t2` = `0xabcd1234` and `$t3` = 16

```
srlv $t1, $t2, $t3        # $t1 = 0x________
```

Op	rs	$t2	$t1	sa	srlv
000000	_____	_____	_____	_____	_____



Logic Bitwise Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func
`and $t1, $t2, $t3`	`$t1 = $t2 & $t3`	0	$t2	$t3	$t1	0	0x24
`or  $t1, $t2, $t3`	`$t1 = $t2 | $t3`	0	$t2	$t3	$t1	0	0x25
`xor $t1, $t2, $t3`	`$t1 = $t2 ^ $t3`	0	$t2	$t3	$t1	0	0x26
`nor $t1, $t2, $t3`	`$t1 = ~($t2 | $t3)`	0	$t2	$t3	$t1	0	0x27

- Examples:
- Given: $t1 = 0xabcdf214 and $t2 = 0xffff0201

```
and $t0, $t1, $t2        # $t0 = 0x_________
or  $t0, $t1, $t2        # $t0 = 0x_________
xor $t0, $t1, $t2        # $t0 = 0x_________
nor $t0, $t1, $t2        # $t0 = 0x_________
```



Binary
Multiplication

Binary Multiplication

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- **Shift Left Instruction** (**`sll`**) can perform multiplication
  - When the **multiplier** is a power of 2
- You can factor any binary number into powers of 2
- Example: multiply **`$t0`** by **`36`**

```
$t0*36 = $t0*(4 + 32) = $t0*4 + $t0*32

  sll  $t1, $t0, 2     # $t1 = $t0 * 4
  sll  $t2, $t0, 5     # $t2 = $t0 * 32
  addu $t3, $t1, $t2   # $t3 = $t0 * 36
```

- 5 in binary: `0000 0101`
- Shift left 1 bit: `0000 1010 = 10` (5×2)
- Shift left 2 bits: `0001 0100 = 20` (5×4)
- Shift left 3 bits: `0010 1000 = 40` (5×8)

Binary Multiplication

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Multiply `$t0` by `26`, using shift and add instructions

```
$t0*26 = $t0*(_____________) = _____________
   sll  $__, $__, __     # $__ = $__ * __
   sll  ...              # ...
   sll  ...              # ...
   addu $__, $__, $__    # $__ = $__ * __
   addu ...              # ...
```



Binary Multiplication

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Multiply `$t0` by `31`, using shift and add instructions

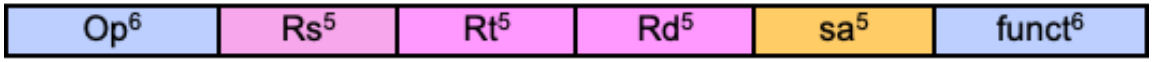
```
$t0*31 = $t0*(_____________) = _____________
   sll  $__, $__, __     # $__ = $__ * __
   sll  ...              # ...
   sll  ...              # ...
   addu $__, $__, $__    # $__ = $__ * __
   addu ...              # ...
```



I-Type Instruction Format

I-Type vs. R-Type Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

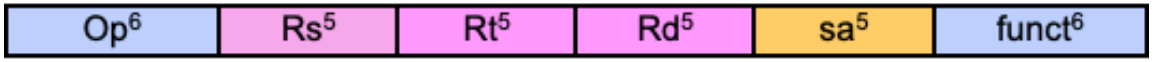
- The key to understanding the instructions is to familiarize yourself with their format.
- All MIPS instructions are 32 bits long, but are divided differently.
- R-Type (Register-Type)
  - Purpose: For operations where all operands are registers, e.g., `add $t1, $t2, $t3`
  - Format: 

  - Why **`Op`** is often `0`:
    - The **`Op`** field (the first 6 bits) is the operation code.
    - For most basic R-type instructions (**`add`**, **`sub`**, **`and`**, **`or`**, etc.), the **`Op`** code is `000000` — this tells the CPU:
      - "Hey, this is an R-type instruction. To find out which R-type instruction, look at the very last 6 bits, the **`funct`** field."

I-Type vs. R-Type Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- The Example: `funct` code `100000` means `add`, `100100` means `and`.
- The other fields are:
  - `Rs`: First source register
  - `Rt`: Second source register
  - `Rd`: Destination register (where the result goes)
  - `sa` or `shamt`: Shift amount (used for shift instructions like `sll`, otherwise `0`)
  - `funct`: Function code (specifies the exact operation)



I-Type vs. R-Type Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- I-Type (Immediate-Type)
  - Purpose: For operations that use a constant value (immediate) embedded in the instruction itself, e.g., `addi $t1, $t2, 25`.
  - Format: 

  - Why **`Op`** is NOT `0`:
    - For I-type instructions, the **`Op`** field uniquely identifies the instruction.
    - There is no **`funct`** field.
    - The CPU sees the **`Op`** code and knows exactly what to do.
- Example: **`Op`** code `001000` means **`addi`**, `001101` means **`ori`**.
    - **`Rs`**: Source register
    - **`Rt`**: Destination register (notice this is different from R-type's **`Rd`**)
    - **`immediate`**: A 16-bit constant value

I-Type Instruction Format

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Constants are used quite frequently in programs.
  - The R-type shift instructions have a 5-bit shift amount constant.
  - What about other instructions that need a constant?
- I-Type: Instructions with **`immediate`** Operands.



- A 16-bit immediate constant is stored inside the instruction
- **`Rs`** is the source register number
- **`Rt`** is now the destination register number (for R-type, it was **`Rd`**)
- Examples of I-Type ALU Instructions:
- Add immediate: `addi $t1, $t2, 5` `# $t1 = $t2 + 5`
- OR immediate: `ori $t1, $t2, 5` `# $t1 = $t2 | 5`

I-Type ALU Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Immediate
`addi $t1, $t2, 25`	`$t1 = $t2 + 25`	0x8	$t2	$t1	25
`addiu $t1, $t2, 25`	`$t1 = $t2 + 25`	0x9	$t2	$t1	25
`andi $t1, $t2, 25`	`$t1 = $t2 & 25`	0xc	$t2	$t1	25
`ori $t1, $t2, 25`	`$t1 = $t2 | 25`	0xd	$t2	$t1	25
`xori $t1, $t2, 25`	`$t1 = $t2 ^ 25`	0xe	$t2	$t1	25
`lui $t1, 25`	`$t1 = 25 << 16`	0xf	0	$t1	25

- **`addi`**: overflow causes an arithmetic exception
  - In case of overflow, the result is not written to the destination register
- **`addiu`**: same operation as **`addi`**, but overflow is ignored
- Immediate constant for **`addi`** and **`addiu`** is signed
  - No need for **`subi`** or **`subiu`** instructions
- Immediate constant for **`andi`**, **`ori`**, **`xori`** is unsigned

I-Type ALU Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Immediate
`addi $t1, $t2, 25`	`$t1 = $t2 + 25`	0x8	$t2	$t1	25
`addiu $t1, $t2, 25`	`$t1 = $t2 + 25`	0x9	$t2	$t1	25
`andi $t1, $t2, 25`	`$t1 = $t2 & 25`	0xc	$t2	$t1	25
`ori $t1, $t2, 25`	`$t1 = $t2 | 25`	0xd	$t2	$t1	25
`xori $t1, $t2, 25`	`$t1 = $t2 ^ 25`	0xe	$t2	$t1	25
`lui $t1, 25`	`$t1 = 25 << 16`	0xf	0	$t1	25

- `lui $t1, 0x1234` loads the 16-bit `immediate` value into the upper half (the most significant 16 bits) of the destination register `$t1` and sets the lower 16 bits to zero.
- Result: `$t1` becomes `0x12340000`.
- Since the `immediate` field in I-type instructions is only 16 bits, we can't load a full 32-bit constant (like a memory address) in one instruction.

I-Type ALU Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Instruction	Meaning	Op	Rs	Rt	Immediate
`addi $t1, $t2, 25`	`$t1 = $t2 + 25`	0x8	$t2	$t1	25
`addiu $t1, $t2, 25`	`$t1 = $t2 + 25`	0x9	$t2	$t1	25
`andi $t1, $t2, 25`	`$t1 = $t2 & 25`	0xc	$t2	$t1	25
`ori $t1, $t2, 25`	`$t1 = $t2 | 25`	0xd	$t2	$t1	25
`xori $t1, $t2, 25`	`$t1 = $t2 ^ 25`	0xe	$t2	$t1	25
`lui $t1, 25`	`$t1 = 25 << 16`	0xf	0	$t1	25

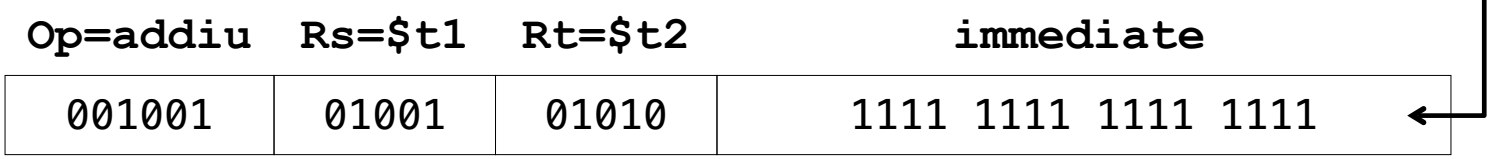
- `lui` is the first step of a two-step process:
  - `lui $t1, 0x1001` # Load the upper part of the address: `$t1 = 0x10010000`
  - `ori $t1, $t1, 0x1234`
    # OR in the lower part: `$t1 = 0x10010000 | 0x00001234 = 0x10011234`

- Now `$t1` holds the full 32-bit address `0x10011234`.

Examples of I-Type ALU Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Expression	Equivalent MIPS Instruction
`A = B + 5;`	`addiu $t0, $t1, 5`
`C = B - 1;`	`addiu $t2, $t1, -1`
`A = B & 0xf;`	`andi  $t0, $t1, 0xf`
`C = B | 0xf;`	`ori   $t2, $t1, 0xf`
`C = 5;`	`addiu $t2, $zero, 5`
`A = B;`	`addiu $t0, $t1, 0`



- No need for **`subiu`**, because **`addiu`** has signed **`immediate`**
- Register `$zero` has always the value `0`

MIPS Instruction Types

Type	-31- format (bits) -0-					
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

R-Type Instructions (Opcode 000000)

Main processor instructions that do not require a target address, immediate value, or branch displacement use an R-type coding format. This format has fields for specifying of up to three registers and a shift amount.

For instructions that do not use all of these fields, the unused fields are coded with all 0 bits. All R-type instructions use a 000000 opcode. The operation is specified by the function field.

opcode (6)	rs (5)	rt (5)	rd (5)	sa (5)	function (6)

Instruction		Function
add	rd, rs, rt	100000
addu	rd, rs, rt	100001
and	rd, rs, rt	100100
break		001101
div	rs, rt	011010
divu	rs, rt	011011
jalr	rd, rs	001001
jr	rs	001000
mfhi	rd	010000
mflo	rd	010010
mthi	rs	010001
mtlo	rs	010011
mult	rs, rt	011000
multu	rs, rt	011001
nor	rd, rs, rt	100111
or	rd, rs, rt	100101
sll	rd, rt, sa	000000
sllv	rd, rt, rs	000100
slt	rd, rs, rt	101010
sltu	rd, rs, rt	101011
sra	rd, rt, sa	000011
srav	rd, rt, rs	000111
srl	rd, rt, sa	000010
srlv	rd, rt, rs	000110
sub	rd, rs, rt	100010
subu	rd, rs, rt	100011
syscall		001100
xor	rd, rs, rt	100110

s of I-Type ALU Instruction

Bitwise, Shift, and I-Type Instructions

Expression	Equivalent MIPS Instruction
`A = B + 5;`	`addiu $t0, $t1, 5`
`C = B - 1;`	`addiu $t2, $t1, -1`
`A = B & 0xf;`	`andi $t0, $t1, 0xf`
`C = B | 0xf;`	`ori $t2, $t1, 0xf`
`C = 5;`	`addiu $t2, $zero, 5`
`A = B;`	`addiu $t0, $t1, 0`

Op=addiu	Rs=$t1	Rt=$t2	immediate
001001	01001	01010	1111 1111 1111 1111



10 minutes



I-Type Instructions (All opcodes except 000000, 00001x, and 0100xx)

I-type instructions have a 16-bit immediate field that codes an immediate operand, a branch target offset, or a displacement for a memory operand. For a branch target offset, the immediate field contains the signed difference between the address of the following instruction and the target label, with the two low order bits dropped. The dropped bits are always 0 since instructions are word-aligned.

For the `bgez`, `bgtz`, `blez`, and `bltz` instructions, the rt field is used as an extension of the opcode field.

opcode (6)	rs (5)	rt (5)	immediate (16)

Instruction		Opcode	Notes
addi	rt, rs, immediate	001000	
addiu	rt, rs, immediate	001001	
andi	rt, rs, immediate	001100	
beq	rs, rt, label	000100	
bgez	rs, label	000001	rt = 00001
bgtz	rs, label	000111	rt = 00000
blez	rs, label	000110	rt = 00000
bltz	rs, label	000001	rt = 00000
bne	rs, rt, label	000101	
lb	rt, immediate(rs)	100000	
lbu	rt, immediate(rs)	100100	
lh	rt, immediate(rs)	100001	
lhu	rt, immediate(rs)	100101	
lui	rt, immediate	001111	
lw	rt, immediate(rs)	100011	
lwc1	rt, immediate(rs)	110001	
ori	rt, rs, immediate	001101	
sb	rt, immediate(rs)	101000	
slti	rt, rs, immediate	001010	
sltiu	rt, rs, immediate	001011	
sh	rt, immediate(rs)	101001	
sw	rt, immediate(rs)	101011	
swc1	rt, immediate(rs)	111001	
xori	rt, rs, immediate	001110	

32-bit Constants

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- I-Type instructions can have only 16-bit constants



- What if we want to load a 32-bit constant into a register?
- Can't have a 32-bit constant in I-Type instructions ☹
  - The sizes of all instructions are fixed to 32 bits
- Solution: use two instructions instead of one ☺
- Suppose we want: `$t1 = 0xAC5165D9` (32-bit constant)

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```

	Upper 16 bits	Lower 16 bits
`# $t1`	`0xAC51`	`0x0000`
`# $t1`	`0xAC51`	`0x65D9`

32-bit Constants

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- The Solution: Two Instructions (**`lui`** + **`ori`**)



- To load a 32-bit number, we use two separate instructions:
  - One to set the upper 16 bits of the target register.
  - One to set the lower 16 bits of the target register.

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```

	Upper 16 bits	Lower 16 bits
`# $t1`	`0xAC51`	`0x0000`
`# $t1`	`0xAC51`	`0x65D9`

32-bit Constants

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

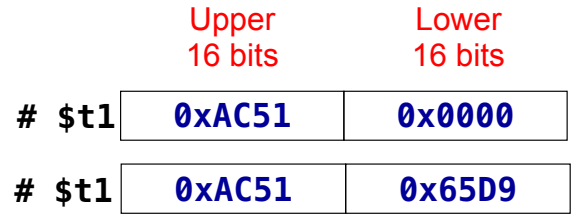
- The Solution: Two Instructions (**`lui`** + **`ori`**)



- The **`lui`** (Load Upper Immediate) Instruction
- Purpose: **`lui`** `$rt`, immediate loads a 16-bit immediate value into the upper 16 bits of the target register ($rt).
  - It then sets the lower 16 bits to zero.
  - The upper half (`AC51`) is in place, and the lower half is zeros (`0000`).

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```



32-bit Constants

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

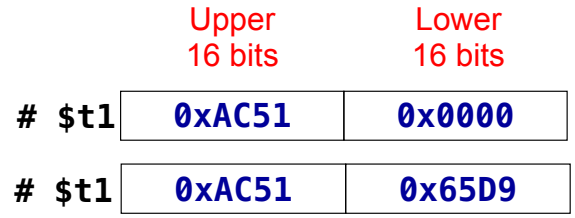
- The Solution: Two Instructions (**`lui`** + **`ori`**)



- Completing the Load with **`ori`** (Or Immediate)
- The **`lui`** instruction zeroed out the lower 16 bits.
  - We now need to put our desired lower 16 bits (`0x65D9`) into the register without disturbing the upper bits we just set.

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```



32-bit Constants

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

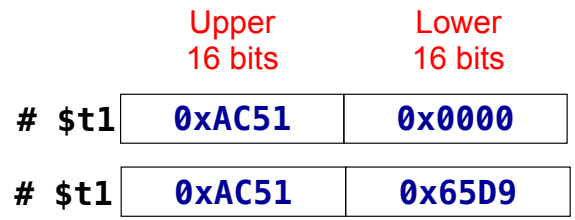
- The Solution: Two Instructions (**`lui`** + **`ori`**)



- We use the **`ori`** (Or Immediate) instruction for this.
  - The logical OR operation is perfect:
    `0xAC510000 OR 0x000065D9 = 0xAC5165D9`
- The instruction would be: `ori $t1, $t1, 0x65D9`

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```



32-bit Constants

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

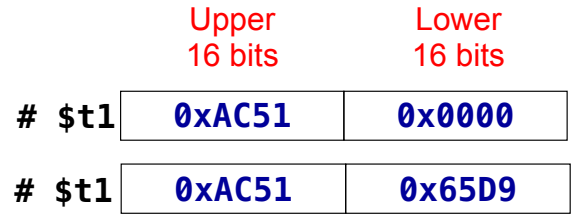
- The Solution: Two Instructions (**`lui`** + **`ori`**)



- This takes the current value of `$t1` (`0xAC510000`), ORs it with `0x65D9`, and stores the result (`0xAC5165D9`) back into `$t1`.

`lui`: load upper immediate

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```



Pseudo Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

- Introduced by the assembler as if they were real instructions.
- Facilitate assembly language programming.

Pseudo-Instruction	Equivalent MIPS Instruction
`move $t1, $t2`	`addu $t1, $t2, $zero`
`not $t1, $t2`	`nor $t1, $t2, $zero`
`neg $t1, $t2`	`sub $t1, $zero, $t2`
`li $t1, -5`	`addiu $t1, $zero, -5`
`li $t1, 0xabcd1234`	`lui $t1, 0xabcd`
	`ori $t1, $t1, 0x1234`

- The MARS tool has a long list of pseudo-instructions.

Pseudo Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

What are Pseudo-Instructions?

- They are instructions that do not exist in the actual MIPS hardware instruction set.
- They are provided by the assembler (the program that converts text code into machine code) as a convenience for the programmer.
- The assembler sees a pseudo-instruction and automatically replaces it with one or more real, valid MIPS instructions.

Pseudo Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Why are they useful?

- They make code easier to read and write.
- They act as helpful shortcuts, saving the programmer from writing longer, more complex sequences of real instructions.

Pseudo Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Examples from the Table: `move $t1, $t2`

- Meaning: Copy the value from register `$t2` into register `$t1`.
- Implementation: `addu $t1, $t2, $zero`
- How it works: It adds the value in `$t2` to the value in `$zero` (which is always `0`) and stores the result in `$t1`.
    - This is effectively a copy operation.

Pseudo Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Examples from the Table: `not $t1, $t2`

- Meaning: Perform a bitwise logical NOT on the value in `$t2` (flip all bits) and store the result in `$t1`.
- Implementation: `nor $t1, $t2, $zero`
- How it works: NOR is “NOT OR”.
    - The operation A NOR `0` is equivalent to NOT A.
    - This is a clever way to achieve a NOT operation.

Pseudo Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Examples from the Table: `neg $t1, $t2`

- Meaning: Negate the value in `$t2` (using two's complement) and store the result in `$t1`.
- Implementation: `sub $t1, $zero, $t2`
- How it works: It subtracts `$t2` from 0, which is the definition of arithmetic negation.

Pseudo Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Examples from the Table: `li $t1, -5`

- Load Immediate
- Meaning: Load the small constant -5 into register `$t1`.
- Implementation: `addiu $t1, $zero, -5`
- How it works: It adds the sign-extended 16-bit value -5 to `$zero` and stores it in `$t1`.

Pseudo Instructions

Unit 7: Logic Bitwise, Shift, and I-Type Instructions

Examples from the Table: `li $t1, 0xabcd1234`

- Load Immediate with a 32-bit constant
- This is the direct application of the concept for a 32-bit constant.
- Meaning: Load the large 32-bit constant into `$t1`.
- Implementation:

```
lui $t1, 0xAC51
ori $t1, $t1, 0x65D9
```

	Upper 16 bits	Lower 16 bits
`# $t1`	`0xAC51`	`0x0000`
`# $t1`	`0xAC51`	`0x65D9`

References

References

Reference Materials

- William Stallings, “Computer Organization and Architecture Designing for Performance,” Tenth Edition, *Pearson*, 2016
- Aiman El-Maleh, “Computer Organization: Lectures,” https://faculty.kfupm.edu.sa, accessed August 26, 2025



